

INSTITUTO FEDERAL

Mato Grosso do Sul

Construção de Páginas Web III

Milena Alegre

POST

Método Http para **criação** de novos dados

GET → buscar dados

POST → criar dados

PUT → atualizar tudo

DELETE → remover

Para conseguir utilizar o POST nós temos que habilitar o JSON.

```
app.use(express.json());
```

POST

POSTMAN = O Postman é uma plataforma colaborativa para desenvolvimento e testes de APIs, com recursos robustos para enviar requisições HTTP, validar respostas e automatizar cenários.

Ele conta com uma interface gráfica moderna e intuitiva, que facilita desde o uso por iniciantes até times que trabalham com integração contínua.



POST

Implementação

```
//post
app.post("/filmes", (req, res) => {
  const novoFilme = req.body;
  filmes.push(novoFilme);

  res.json({
    mensagem: "Filme adicionado!",
    filme: novoFilme
  });
});
```

POST

Criar requisição

- Abre o Postman
- Cria uma nova Coleção
- Clica em New → HTTP Request
- Seleciona o Método **POST**

URL: <http://localhost:{PORTA}/filmes>

- Ir na aba Body
- Seleciona raw
- Do lado escolhe JSON

POST

Adicionar o ID manualmente é perigoso!!!

Como resolver?

```
app.post("/filmes", (req, res) => {  
  const { titulo, ano, nota } = req.body;  
  
  const novoFilme = {  
    id: filmes.length + 1,  
    titulo,  
    ano,  
    nota  
  };  
  
  filmes.push(novoFilme);  
  
  res.json(novoFilme);  
});
```

POST

Teste = adicionando novo filme

Método = POST

URL: `http://localhost:{PORTA}/filmes`

```
{  
  "id": 6,  
  "titulo": "Matrix",  
  "ano": 1999,  
  "nota": 10  
}
```



POST

Alternativa para o postman? extensao do VScode

Thunder Client

PUT

Método Http para **atualizar** dados

GET → buscar dados

POST → criar dados

PUT → atualizar tudo

DELETE → remover

Diferente do **POST** (que cria), o **PUT** altera algo que já existe.

PUT

Implementação

```
//put
app.put("/filmes/:id", (req, res) => {
  const id = parseInt(req.params.id);
  const { titulo, ano, nota } = req.body;

  const filme = filmes.find(f => f.id === id);

  if (!filme) {
    return res.send("Filme não encontrado");
  }

  // atualizando os dados
  filme.titulo = titulo;
  filme.ano = ano;
  filme.nota = nota;

  res.json({
    mensagem: "Filme atualizado com sucesso!",
    filme: filme
  });
});
```

PUT

Teste = Atualizando filme

Método = PUT

URL: `http://localhost:{PORTA}/filmes/{id}`

```
{  
  "titulo": "A Origem",  
  "ano": 2010,  
  "nota": 10000  
}
```



DELETE

Método Http para **apagar** dados

GET → buscar dados

POST → criar dados

PUT → atualizar tudo

DELETE → remover

DELETE

Implementação

```
//delete
app.delete("/filmes/:id", (req, res) => {
  const id = parseInt(req.params.id);

  const novaLista = filmes.filter(f => f.id !== id);

  if (novaLista.length === filmes.length) {
    return res.send("Filme não encontrado");
  }

  // atualiza o array original
  filmes.length = 0;
  filmes.push(...novaLista);

  res.json({
    mensagem: "Filme removido com sucesso!"
  });
});
```

DELETE

Teste = Apagando filme

Método = DELETE

URL: `http://localhost:{PORTA}/filmes/{id}`

NÃO precisa de Body

Diferente do POST e PUT Não precisa clicar em Body
Não precisa mandar JSON



Relembrando



CREATE

C



READ

R



UPDATE

U



DELETE

D

Relembrando

- Comandos SQL	Verbos do	-
-	Protocolo HTTP	-

- CREATE	POST	-
- READ	GET	-
- UPDATE	PUT	-
- DELETE	DELETE	-

Iniciando

valores padroes

```
npm init -y  
npm install express  
  
node server.js
```

Fetch

fetch é uma função do JavaScript usada para: fazer requisições HTTP (GET, POST, etc) para APIs

fetch = Ir buscar dados em algum lugar (geralmente um servidor).

Exemplo básico (GET)

```
fetch("https://dog.ceo/api/breeds/image/random")
```

Mas o fetch não entrega direto o dado!

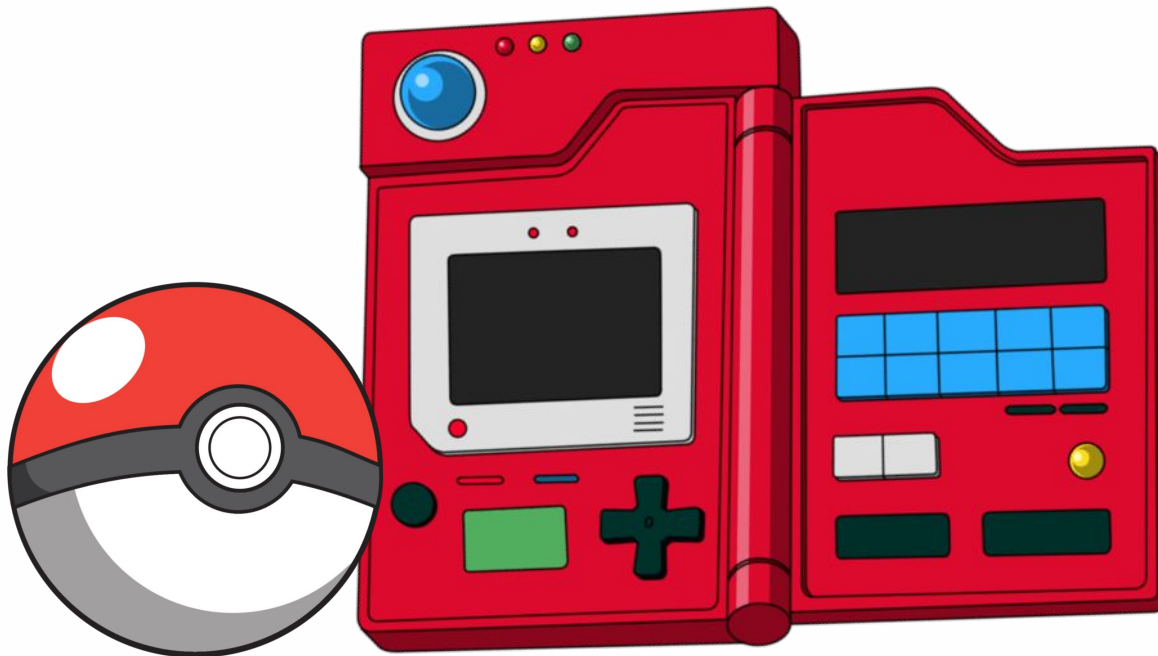
```
fetch("https://dog.ceo/api/breeds/image/random")  
  .then(res => res.json())  
  .then(dados => {  
    console.log(dados);  
  });
```

Fetch

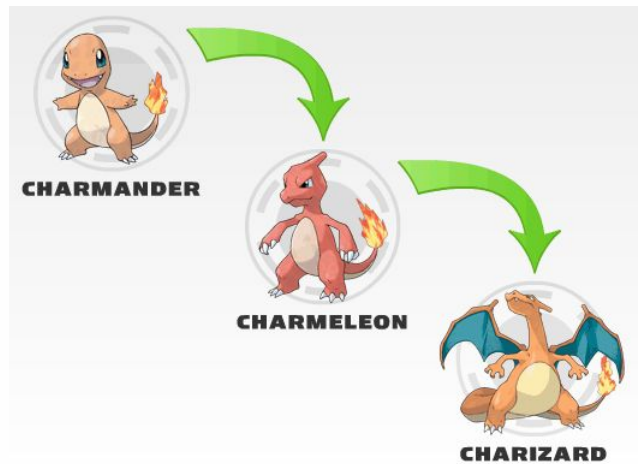
```
app.get("/dog", async (req, res) => {  
  const response = await fetch("https://dog.ceo/api/breeds/image/random");  
  const dados = await response.json();  
  
  res.json(dados);  
});  
  
const PORTA = 3000;  
  
app.listen(PORTA, () => {  
  console.log(`Servidor rodando em http://localhost:${PORTA}`);  
});
```

Pokédex

O que é?



Evolução:



Consumo API

Qual formato que a API devolve?

Importante:

`{}` → objeto

`:` → separa chave e valor

`,` → separa itens

Tipos de dados importantes

Objeto: `dados.name`

Lista (array): `dados.types`

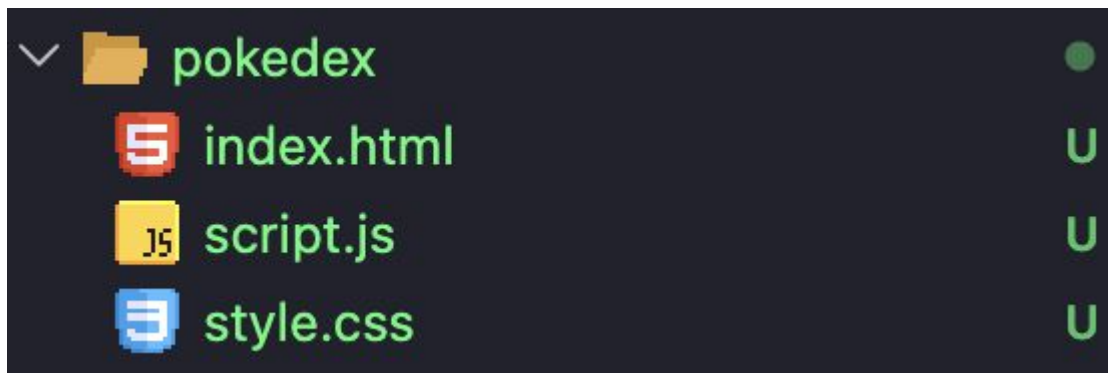
Se for lista → precisa de loop (map ou forEach)

map percorre a lista e cria outra lista nova
`[1, 2, 3].map(n => n * 2)`
`// resultado: [2, 4, 6]`

Consumo API

<https://pokeapi.co/api/v2/pokemon/pikachu>

Estrutura pastas:



Consumo API

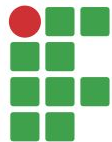
Exemplo da api:

```
const lista = p.abilities;
```

```
const nomes = lista.map(a => a.ability.name);
```

```
const texto = nomes.join(", ");
```


milena.alegre@ifms.edu.br



INSTITUTO FEDERAL
Mato Grosso do Sul